GenSpark

Input, Output, Type Conversion, and Checking Text

In the last article you stored values in variables and displayed them with `print()`. Real programs also need to take in information from a person and make sure it is usable before relying on it. This article covers four skills: reading input, converting types, formatting output, and checking text. Together they are the first half of what the Number Guessing Game needs.

Reading Input with input()

The built-in `input()` function pauses your program, shows a prompt, and waits for the user to type something and press Enter. Whatever they typed comes back as a value you can store in a variable.

python

```
name = input("What's your name? ")
print(f"Hello, {name}!")
```

The text inside the parentheses is the prompt. Put a space at the end of it, so the user's typing doesn't touch your question.

input() Always Returns a String

This is the most important fact in this article. Everything `input()` returns is a string, even when the user types a number.

python

```
guess = input("Pick a number: ")    # the user types 7
print(type(guess))                  # <class 'str'>
```

Here `guess` holds the text `"7"`, not the number 7. That causes trouble when you try to use it as a number:

python

```
guess = input("Pick a number: ")    # the user types 7
print(guess + 1)                    # TypeError: can only concatenate str (not
```

Python won't add text and a number. You have to convert the text first.

Type Conversion

Python gives you a function for each type that converts a value into that type.

Function	Converts to	Example	Result
`int()`	Whole number	`int("7")`	`7`
`float()`	Decimal number	`float("3.5")`	`3.5`
`str()`	Text	`str(42)`	`"42"`

Convert the input, then use it:

python

```
guess = int(input("Pick a number: "))    # the user types 7
print(guess + 1)                         # 8
```

You can also convert in two steps, which is easier to read while you're learning:

python

```
text = input("Pick a number: ")
guess = int(text)
```

Some other conversions to know:

python

```
int(3.9)        # 3, the decimal part is dropped, not rounded
float(5)        # 5.0
str(3.14)       # "3.14"
int("42") + 8   # 50
str(42) + "8"   # "428", two strings joined together
```

Notice the last two lines. The `+` operator adds numbers, but it joins strings together. This is why the type matters.

When Conversion Fails

If the text can't become a number, `int()` stops your program with a `ValueError`.

python

```
age = int(input("How old are you? "))    # the user types "twenty"


ValueError: invalid literal for int() with base 10: 'twenty'
```

Real users type letters, leave the answer blank, or press Enter by accident. A program that crashes on bad input is not ready to use. The fix is to check the text before you convert it, which is the last section of this article.

Two details to remember:

- `int("3.5")` also fails, because `"3.5"` is not a whole number. Use `float("3.5")` instead.
- `int(" 7 ")` works, because Python ignores the spaces around a number. Still, clean your input as shown below.

Formatting Output

You already know `print()` and f-strings. Here are a few more tools that make output clear.

Printing several items. Commas add a space between items:

python

```
print("Attempts left:", 5)    # Attempts left: 5
```

Putting values in a sentence. An f-string lets you embed any value or calculation:

python

```
attempts_left = 8
print(f"You have {attempts_left} attempts left.")
print(f"After this guess you will have {attempts_left - 1} left.")
```

Controlling the ending. `print()` normally ends with a new line. Change that with `end`:

python

```
print("Loading", end="")
print("...")                  # Loading...
```

Blank lines and special characters. `\n` inside a string starts a new line:

python

```
print("Line one\nLine two")
```

Joining with a separator. The `sep` option changes the space between items:

python

```
print("2026", "10", "02", sep="-")    # 2026-10-02
```

Working with Strings

A string is a sequence of characters. Python has many built-in tools, called methods, for working with them. A method is a function attached to a value, and you call it with a dot.

python

```
text = "  Hello World  "
print(text.strip())      # "Hello World", removes spaces from both ends
print(text.lower())      # "  hello world  "
print(text.upper())      # "  HELLO WORLD  "
print(len(text))         # 15, the number of characters
```

One rule catches almost every beginner: **string methods don't change the original.** They return a new string. To keep the result, store it:

python

```
text = "  Hello World  "
text.strip()             # does nothing useful, the result is thrown away
text = text.strip()      # now text holds "Hello World"
```

You can chain methods, and you can clean input in one line:

python

```
answer = input("Continue? ").strip().lower()
```

This removes extra spaces and makes the text lowercase, so `" YES "` and `"yes"` become the same value.

Checking Text

Before you convert text to a number, you need to know whether it is safe to convert. These tools answer yes or no questions about a string. Each one returns `True` or `False`.

isdigit() is `True` when every character is a digit:

python

```
"42".isdigit()       # True
"4x2".isdigit()      # False
"".isdigit()         # False, an empty string has no digits
"-5".isdigit()       # False, the minus sign is not a digit
"3.5".isdigit()      # False, the dot is not a digit
```

Because it returns `False` for blank text, letters, symbols, and decimals, `isdigit()` is a good check for "is this a plain whole number I can pass to `int()`?" It doesn't accept negative numbers, which is fine when your program only expects positive ones.

`isdigit()` also returns `True` for a few unusual characters, such as the superscript `²`, that `int()` cannot convert. Typed keyboard digits never cause this, so `isdigit()` is safe for this course. In production code, wrap the conversion in `try` and `except`, which you learn in Week 3.

The `in` operator checks whether one value appears inside another:

python

```
"an" in "banana"              # True
"z" in "banana"               # False
"2" in ["1", "2", "3"]        # True, checking membership in a list
```

The list version is how you check a menu choice. If the valid answers are `"1"`, `"2"`, and `"3"`, then `choice in ["1", "2", "3"]` tells you whether the user's answer is one of them.

Comparing text with `==` asks whether two strings are exactly the same:

python

```
"yes" == "yes"    # True
"yes" == "Yes"    # False, capital letters count
"yes" == "yes "   # False, the extra space counts
```

That is why you clean first with `strip()` and `lower()`, and compare second.

Other useful checks:

Check	Meaning	Example	Result
`.isalpha()`	Only letters	`"abc".isalpha()`	`True`
`.startswith(x)`	Begins with x	`"Hello".startswith("He")`	`True`
`.endswith(x)`	Ends with x	`"file.py".endswith(".py")`	`True`
`len(text) == 0`	Is empty	`len("") == 0`	`True`

Putting It Together

Here is a short program that uses every idea in this article:

python

```
# Ask for a name and clean it up
name = input("What's your name? ").strip().title()

# Read an age as text, then check it
age_text = input("How old are you? ").strip()
print(f"Is that a whole number? {age_text.isdigit()}")

# Once we know it is safe, convert and use it
age = int(age_text)
print(f"Hello, {name}! Next year you will be {age + 1}.")
```

If the user types a number for the age, the program works. If they type `"twenty"`, it prints `False` and then crashes on `int(age_text)`. You can see the problem, but you can't yet act on the `False`. In the next article you learn to make decisions with `if` statements, and those let you ask again instead of crashing. In the article after that, loops let you keep asking until the answer is valid. Together, those three skills are the heart of the Number Guessing Game.

Common Mistakes

Mistake	What happens	Fix

Using `input()` text as a number	`TypeError` when doing math	Convert with `int()` or `float()`
Calling `int()` on bad text	`ValueError` crash	Check with `isdigit()` first
Writing `text.strip()` and not saving it	The text stays unchanged	Assign: `text = text.strip()`
Comparing without cleaning	`"Yes "` does not equal `"yes"`	Use `.strip().lower()` first
Forgetting the quotes on a number	`"5"` is text, `5` is a number	Check with `type()`
Missing the space at the end of a prompt	Typing runs into the prompt	End the prompt with a space

Try It Yourself

Create a file called `practice.py` in your project, run it with `python3 practice.py`, and try these:

1. Ask for the user's name and print a greeting. Clean the name with `strip()` and `title()`.
2. Ask for two numbers, convert both with `int()`, and print their sum.
3. Ask for a word, then print the word in uppercase and the number of letters in it.
4. Ask for a number and print whether the text `isdigit()`. Try `42`, `4x2`, a blank entry, `-5`, and `3.5`.
5. Ask for a menu choice of 1, 2, or 3, and print whether `choice in ["1", "2", "3"]` is `True`.

Run each one a few times with bad input. Seeing what breaks is how this sticks.

Why This Matters for an FDE

Software on a client engagement spends much of its time receiving data it does not control: user entries, file contents, API responses, and spreadsheet cells. That data arrives as text, in the wrong format, with extra spaces, or missing. A habit of cleaning, checking, and converting input before using it is what separates code that works in a demo from code that survives real use.

Comprehension Check

1. What type does `input()` always return? What happens if you run `input("Number: ") + 1`?
2. What is the difference between `"5" + "3"` and `int("5") + int("3")`?
3. What does `" Yes ".strip().lower()` return? Why would you do this before comparing the answer to `"yes"`?
4. For each of these, does `isdigit()` return `True` or `False`: `"250"`, `"2 5"`, `""`, `"-3"`?
5. You write `name.strip()` on its own line, but `name` still has its spaces. Why, and how do you fix it?

← Previous: Python Foundations: Virtual Environments, History, and the Basics

Next: Control Flow: Decisions with if, elif, and else →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!